

PENJELASAN LENGKAP NOTEBOOK UTAMA

project-deep-learning-lstm-rainfall-in-india.ipynb

BAGIAN 1: SETUP DAN IMPORT LIBRARY

Cell 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
import warnings
import calendar

warnings.filterwarnings('ignore')

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
```

Penjelasan Library:

Library	Fungsi
pandas	Manipulasi data tabular (DataFrame)
numpy	Operasi numerik dan array
matplotlib.pyplot	Visualisasi grafik dasar
seaborn	Visualisasi statistik lanjutan
datetime	Manipulasi tanggal dan waktu
calendar	Informasi kalender (hari per bulan)
tensorflow	Framework deep learning
Sequential	Model neural network berurutan
LSTM	Layer Long Short-Term Memory
Dense	Layer fully connected

Library	Fungsi
Dropout	Layer regularisasi
EarlyStopping	Callback untuk stop training otomatis
ModelCheckpoint	Callback untuk simpan model terbaik
MinMaxScaler	Normalisasi data ke range 0-1
train_test_split	Membagi data train/test
r2_score	Menghitung R ² score

GPU Check:

```
print("Num GPUs Available: ", len(tf.config.list_physical_devices('GPU')))
```

- Mengecek apakah GPU tersedia untuk training
- Jika ada GPU, training akan lebih cepat

BAGIAN 2: CLASS DATALOADER

Cell: Class DataLoader

```
class DataLoader:
    def __init__(self, data_path='rainfall in india 1901-2015.csv'):
        self.data_path = data_path
        self.data = None
        self.scaler = MinMaxScaler(feature_range=(0, 1))
```

Penjelasan `__init__`:

- `data_path`: Path ke file CSV dataset
- `data`: Variabel untuk menyimpan DataFrame
- `scaler`: MinMaxScaler untuk normalisasi ke range 0-1

Apa itu MinMaxScaler? MinMaxScaler adalah algoritma normalisasi yang mengubah data ke range tertentu (default 0-1).

Rumus MinMaxScaler:

$$x_{\text{scaled}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$$

Contoh Konkret:

```
Data asli: [100, 200, 300, 400, 500] mm
x_min = 100, x_max = 500
```

Perhitungan:

- 100: $(100-100)/(500-100) = 0/400 = 0.00$
- 200: $(200-100)/(500-100) = 100/400 = 0.25$
- 300: $(300-100)/(500-100) = 200/400 = 0.50$
- 400: $(400-100)/(500-100) = 300/400 = 0.75$
- 500: $(500-100)/(500-100) = 400/400 = 1.00$

```
Hasil scaled: [0.00, 0.25, 0.50, 0.75, 1.00]
```

Mengapa perlu normalisasi?

1. Neural network bekerja optimal dengan nilai 0-1
2. Mencegah dominasi fitur dengan nilai besar
3. Mempercepat konvergensi training (gradient descent lebih stabil)
4. Menghindari masalah numerik (overflow/underflow)

Method: `load_data()`

```
def load_data(self):
    self.data = pd.read_csv(self.data_path)
    print(f"Dataset loaded with shape: {self.data.shape}")
    return self.data
```

Fungsi: Membaca file CSV ke DataFrame **Output:** DataFrame dengan shape (4116, 19)

Method: `preprocess_data()`

```
def preprocess_data(self, subdivision=None):
    monthly_cols = ['JAN', 'FEB', 'MAR', 'APR', 'MAY', 'JUN',
                    'JUL', 'AUG', 'SEP', 'OCT', 'NOV', 'DEC']

    # Encode subdivision ke angka
    encoder = LabelEncoder()
    self.data['SUB_ID'] = encoder.fit_transform(self.data['SUBDIVISION'])

    # Handle missing values
    data_filtered = data_filtered.fillna(method='ffill').fillna(method='bfill')

    # Konversi bulanan ke harian
    for day in range(1, days_in_month + 1):
        daily_rainfall = rainfall / days_in_month
```

Langkah-langkah Detail:**1. LabelEncoder**

Apa itu? Algoritma yang mengubah data kategorikal (string) menjadi angka.

Mengapa perlu? Machine learning hanya bisa memproses angka, bukan string.

Contoh:

Input (string):		Output (angka):
'ANDAMAN & NICOBAR ISLANDS'	→	0
'ARUNACHAL PRADESH'	→	1
'ASSAM & MEGHALAYA'	→	2
'BIHAR'	→	3
...		
'WEST UTTAR PRADESH'	→	35

Proses Internal:

```
encoder = LabelEncoder()
encoder.fit(data['SUBDIVISION']) # Pelajari semua kategori unik
data['SUB_ID'] = encoder.transform(data['SUBDIVISION']) # Ubah ke angka
```

2. fillna (Handle Missing Values)

Apa itu? Metode untuk mengisi nilai yang kosong (NaN/null).

Forward Fill (ffill):

Sebelum:	[10, NaN, NaN, 15, NaN]
Sesudah:	[10, 10, 10, 15, 15]
	↑ ↑ ↑ ↑
	asli copy copy copy

Logika: Isi NaN dengan nilai SEBELUMNYA (maju ke depan).

Backward Fill (bfill):

Sebelum:	[NaN, NaN, 10, NaN, 15]
Sesudah:	[10, 10, 10, 15, 15]
	↑ ↑ ↑
	copy copy copy

Logika: Isi NaN dengan nilai SESUDAHNYA (mundur ke belakang).

Kombinasi ffill + bfill:

```
data.fillna(method='ffill').fillna(method='bfill')
```

- ffill dulu: isi NaN dengan nilai sebelumnya
- bfill kemudian: isi NaN yang tersisa (di awal data) dengan nilai sesudahnya

3. Konversi Bulanan → Harian

Mengapa perlu? LSTM butuh data dengan interval waktu konsisten (harian).

Rumus:

```
daily_rainfall = monthly_rainfall / days_in_month
```

Contoh Detail:

Januari 2015: 310 mm total, 31 hari
 $\text{daily} = 310 / 31 = 10 \text{ mm/hari}$

Setiap hari di Januari mendapat nilai 10 mm:

2015-01-01: 10 mm

2015-01-02: 10 mm

...

2015-01-31: 10 mm

Februari 2015: 56 mm total, 28 hari

$\text{daily} = 56 / 28 = 2 \text{ mm/hari}$

Asumsi: Curah hujan terdistribusi merata dalam satu bulan (simplifikasi).

Output: DataFrame dengan kolom ['SUBDIVISION', 'SUB_ID', 'date', 'rainfall']

Method: `create_sequences()`

```
def create_sequences(self, data, seq_length=30, use_subdivision=True):
    x_all, y_all = [], []

    for sub in subdivisions:
        rainfall_values = sub_df['rainfall'].values.reshape(-1, 1)
        scaled_data = self.scaler.fit_transform(rainfall_values)

        for i in range(len(scaled_data) - seq_length):
            x_all.append(scaled_data[i:i+seq_length])
            y_all.append(scaled_data[i+seq_length])
```

Apa itu Sliding Window? Teknik untuk mengubah data time series menjadi format supervised learning (input → output).

Konsep Dasar:

Data asli: [d1, d2, d3, d4, d5, d6, d7, d8, d9, d10]
 seq_length = 3 (gunakan 3 hari untuk prediksi hari ke-4)

Sliding Window membuat pasangan (X, y):

Window 1: X = [d1, d2, d3] → y = d4
input (3 hari) target (hari ke-4)
Window 2: X = [d2, d3, d4] → y = d5 (geser 1 langkah ke kanan)
Window 3: X = [d3, d4, d5] → y = d6
Window 4: X = [d4, d5, d6] → y = d7

Dengan seq_length=30 (dalam notebook):

Sequence 1: X = [hari 1-30] → y = hari 31
 Sequence 2: X = [hari 2-31] → y = hari 32
 Sequence 3: X = [hari 3-32] → y = hari 33
 ...

Artinya: Model belajar memprediksi curah hujan BESOK
 berdasarkan 30 hari SEBELUMNYA.

Mengapa 30 hari?

- Menangkap pola bulanan curah hujan
- Cukup panjang untuk melihat trend
- Tidak terlalu panjang (efisiensi komputasi)

Proses reshape(-1, 1):

```
rainfall_values = sub_df['rainfall'].values.reshape(-1, 1)
```

Sebelum reshape: [10.5, 8.2, 15.3, 12.1] # Shape: (4,) - 1D array
 Sesudah reshape: [[10.5], [8.2], [15.3], [12.1]] # Shape: (4, 1) - 2D array

Mengapa? MinMaxScaler membutuhkan input 2D (samples × features).

Output Shape Explained:

- **X: (1502262, 30, 1):**
 - 1502262 = jumlah sequences (samples)
 - 30 = timesteps (hari)
 - 1 = features (hanya curah hujan)
- **y: (1502262, 1):**
 - 1502262 = jumlah target values
 - 1 = nilai prediksi (curah hujan hari ke-31)

Method: `split_data()`

```
def split_data(self, X, y, test_size=0.2):
    return train_test_split(X, y, test_size=test_size, shuffle=False)
```

Apa itu train_test_split? Fungsi untuk membagi data menjadi set training dan testing.

Parameter:

- **test_size=0.2:** 80% untuk training, 20% untuk testing
- **shuffle=False:** PENTING untuk time series!

Mengapa shuffle=False untuk Time Series?

Data asli (urut waktu):
[Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec]

shuffle=True (SALAH untuk time series):
Train: [Mar, Aug, Jan, Nov, Feb, Jun, Oct, Apr]
Test: [May, Jul, Sep, Dec]
→ Model bisa "melihat masa depan" saat training!

shuffle=False (BENAR untuk time series):
Train: [Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep]
Test: [Oct, Nov, Dec]
→ Model hanya belajar dari masa lalu, prediksi masa depan

Ilustrasi Split 80-20:

Total: 1,502,262 sequences
├ Training (80%): 1,201,809 sequences (data awal)
└ Testing (20%): 300,453 sequences (data akhir)

Timeline:

|← Training (1901-2006) →|← Test (2006-2015) →|

Method: `inverse_transform()`

```
def inverse_transform(self, scaled_values):
    return self.scaler.inverse_transform(scaled_values)
```

Fungsi: Mengembalikan data dari skala 0-1 ke skala asli (mm)

Rumus Inverse Transform:

$$x_{\text{original}} = x_{\text{scaled}} \times (x_{\text{max}} - x_{\text{min}}) + x_{\text{min}}$$

Contoh:

```
Scaled prediction: 0.75
x_min = 100 mm, x_max = 500 mm

x_original = 0.75 × (500 - 100) + 100
            = 0.75 × 400 + 100
            = 300 + 100
            = 400 mm
```

Mengapa perlu inverse transform?

- Model memprediksi dalam skala 0-1 (karena input di-scale)
- Untuk interpretasi, perlu dikembalikan ke satuan asli (mm)
- Metrics (RMSE, MAE) dihitung dalam satuan asli agar bermakna

BAGIAN 3: CLASS RAINFALLSTM

Cell: Class RainfallSTM

Method: `__init__()`

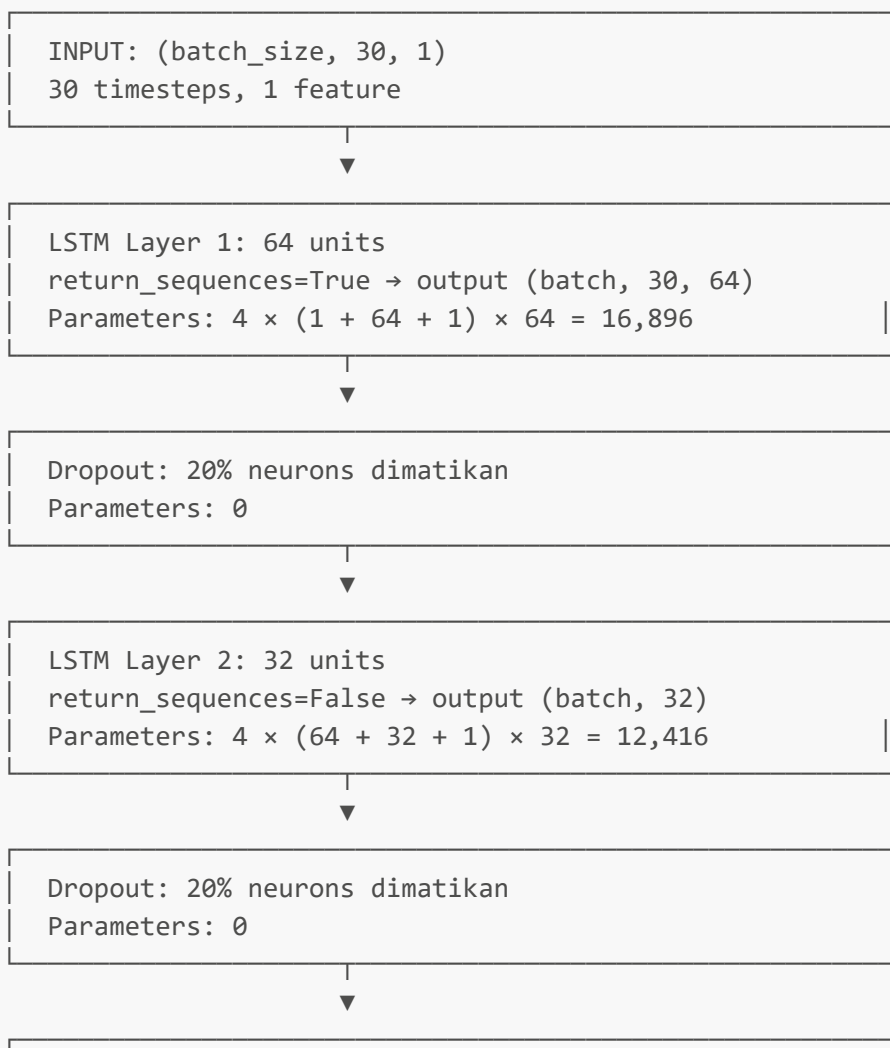
```
def __init__(self, seq_length=30, n_features=1):
    self.seq_length = seq_length
    self.n_features = n_features
    self.model = None
```

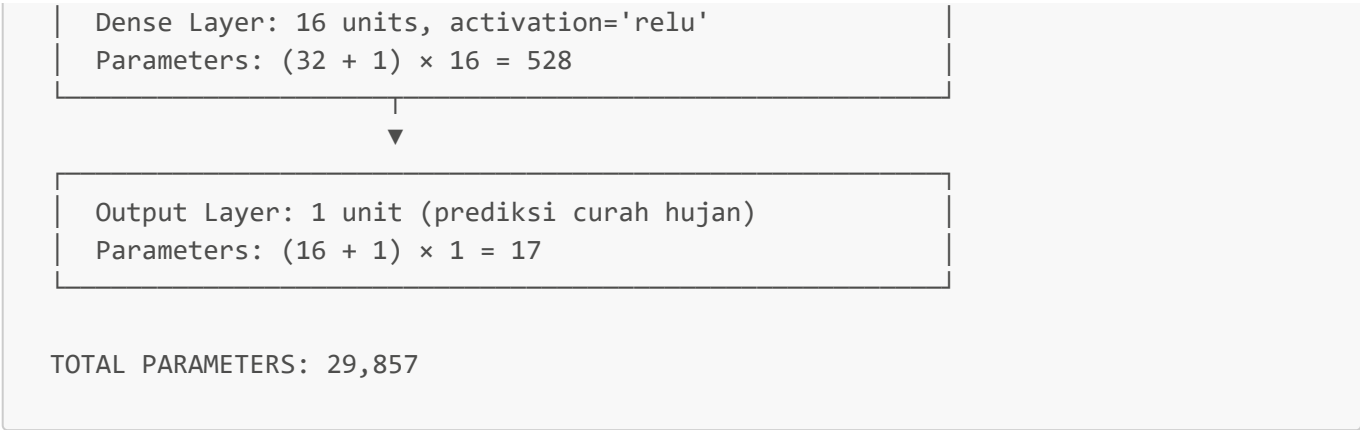

Parameter:

- `seq_length=30`: Panjang sequence input (30 hari)
 - `n_features=1`: Jumlah fitur (hanya curah hujan)
-

Method: `build_model()`

```
def build_model(self, units1=64, units2=32, dense_units=16, dropout_rate=0.2):  
    self.model = Sequential([  
        LSTM(units1, return_sequences=True, input_shape=(self.seq_length,  
self.n_features)),  
        Dropout(dropout_rate),  
        LSTM(units2, return_sequences=False),  
        Dropout(dropout_rate),  
        Dense(dense_units, activation='relu'),  
        Dense(1)  
    ])  
    self.model.compile(optimizer='adam', loss='mean_squared_error')
```

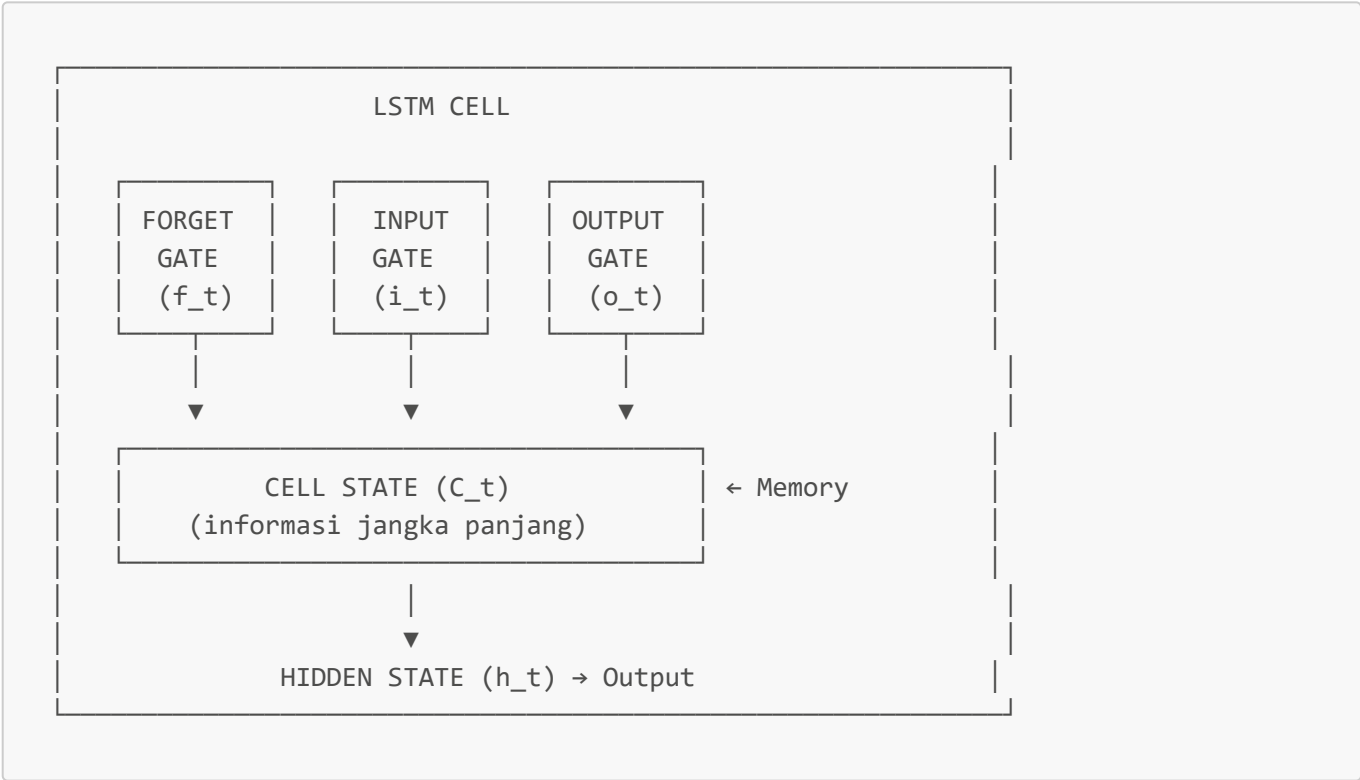
Arsitektur Model:



Apa itu LSTM (Long Short-Term Memory)? LSTM adalah jenis Recurrent Neural Network (RNN) yang dirancang untuk mengingat informasi dalam jangka panjang.

Masalah RNN Biasa: RNN biasa mengalami "vanishing gradient" - tidak bisa mengingat informasi dari timestep yang jauh.

Solusi LSTM: LSTM memiliki "memory cell" dan 3 gate yang mengontrol aliran informasi:



PENJELASAN SIMBOL-SIMBOL MATEMATIKA LSTM:

Simbol	Nama	Penjelasan
t	timestep	Indeks waktu saat ini (hari ke- t)
$t-1$	timestep sebelumnya	Waktu sebelumnya (hari ke- t minus 1)
x_t	input saat ini	Data curah hujan hari ke- t (1 nilai)
h_t	hidden state saat ini	Output LSTM hari ke- t (64 nilai untuk layer 1)

Simbol	Nama	Penjelasan
h_{t-1}	hidden state sebelumnya	Output LSTM hari ke-t-1
C_t	cell state saat ini	Memory jangka panjang hari ke-t
C_{t-1}	cell state sebelumnya	Memory jangka panjang hari ke-t-1
W_f	weight forget gate	Matriks bobot untuk forget gate (lihat penjelasan di bawah)
W_i	weight input gate	Matriks bobot untuk input gate
W_C	weight cell candidate	Matriks bobot untuk kandidat cell
W_o	weight output gate	Matriks bobot untuk output gate
b_f, b_i, b_C, b_o	bias	Nilai konstanta yang ditambahkan (lihat penjelasan di bawah)

Apa itu Weight (W)?

Weight adalah nilai yang menentukan seberapa penting suatu input.

Analogi Sederhana:

Bayangkan Anda menghitung nilai ujian:

Nilai Akhir = $0.3 \times \text{UTS} + 0.4 \times \text{UAS} + 0.3 \times \text{Tugas}$

weight

weight

weight

Weight menentukan "bobot" atau "kepentingan" setiap komponen.

Weight dalam Neural Network:

```
input = [x1, x2, x3] = [0.5, 0.8, 0.2]
weight = [w1, w2, w3] = [0.3, 0.6, 0.1]

output = w1×x1 + w2×x2 + w3×x3
        = 0.3×0.5 + 0.6×0.8 + 0.1×0.2
        = 0.15 + 0.48 + 0.02
        = 0.65
```

Weight Matrix dalam LSTM:

W_f berukuran (64 × 65) untuk LSTM layer 1:

- 64 = jumlah output (units)
- 65 = jumlah input (64 dari h_{t-1} + 1 dari x_t)

Total elemen dalam $W_f = 64 \times 65 = 4,160$ nilai
Setiap nilai dipelajari saat training!

Perbedaan Weight dan Bias:

Aspek	Weight (W)	Bias (b)
Fungsi	Mengalikan input	Ditambahkan ke hasil
Bentuk	Matriks (banyak nilai)	Vektor (satu per neuron)
Analogi	Slope garis	Intercept garis
Jumlah di LSTM	$4 \times 64 \times 65 = 16,640$	$4 \times 64 = 256$
σ	sigmoid	Fungsi aktivasi, output 0-1
$\tanh$	hyperbolic tangent	Fungsi aktivasi, output -1 sampai +1
$[h_{t-1}, x_t]$	concatenation	Gabungan h_{t-1} dan x_t menjadi satu vektor

Apa itu Bias (b)?

Bias adalah nilai konstanta yang ditambahkan ke hasil perkalian $\text{weight} \times \text{input}$.

Analogi Sederhana:

Bayangkan persamaan garis: $y = mx + c$

- m = slope (kemiringan) $\rightarrow$ ini seperti WEIGHT
- c = intercept (titik potong sumbu y) $\rightarrow$ ini seperti BIAS

Tanpa bias ($c=0$): garis HARUS melewati titik $(0,0)$
Dengan bias: garis bisa bergeser ke atas/bawah

Rumus dengan dan tanpa Bias:

TANPA BIAS: $\text{output} = W \times \text{input}$
 $\text{output} = 0.5 \times 2 = 1.0$

DENGAN BIAS: $\text{output} = W \times \text{input} + b$
 $\text{output} = 0.5 \times 2 + 0.3 = 1.3$

$\text{weight} \times \text{input}$

bias

Mengapa Bias Penting?

Contoh: Prediksi curah hujan

Tanpa bias:

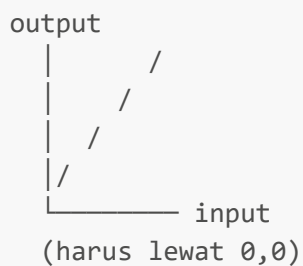
- Jika input = 0, output PASTI = 0
- Model tidak bisa memprediksi "baseline" curah hujan

Dengan bias:

- Jika input = 0, output = bias (bisa > 0)
- Model bisa memprediksi: "Bahkan tanpa pola khusus, rata-rata curah hujan adalah X mm"

Visualisasi:

Tanpa Bias:



Dengan Bias:



Bias dalam LSTM:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + \underbrace{b_f}_{\text{bias}})$$

$b_f = [0.1, -0.2, 0.05, \dots]$ (64 nilai, satu per neuron)

Setiap neuron punya bias sendiri yang dipelajari saat training.

Contoh Konkret dengan Angka:

Misalkan di hari ke-5 ($t=5$):

- $x_5 = 0.75$ (curah hujan hari ke-5, sudah di-scale)
- $h_4 = [0.2, 0.5, 0.1, \dots]$ (64 nilai dari hari ke-4)
- $C_4 = [0.3, 0.8, 0.2, \dots]$ (64 nilai memory dari hari ke-4)

Concatenation $[h_4, x_5]$:

- Gabungkan h_4 (64 nilai) + x_5 (1 nilai) = 65 nilai
- $[0.2, 0.5, 0.1, \dots, 0.75]$

Fungsi Aktivasi:

Sigmoid (σ)

$$\sigma(x) = 1 / (1 + e^{(-x)})$$

Input: $-\infty$ ————— 0 ————— $+\infty$

Output: 0 ————— 0.5 ————— 1

Contoh:

$\sigma(-5) = 0.007 \approx 0$ (hampir lupakan)

$\sigma(0) = 0.5$ (setengah)

$\sigma(5) = 0.993 \approx 1$ (hampir pertahankan)

Kegunaan: Mengontrol "seberapa banyak" (0% sampai 100%)

Tanh (Hyperbolic Tangent)

$$\tanh(x) = (e^x - e^{(-x)}) / (e^x + e^{(-x)})$$

Input: $-\infty$ ————— 0 ————— $+\infty$

Output: -1 ————— 0 ————— $+1$

Contoh:

$\tanh(-5) = -0.9999 \approx -1$

$\tanh(0) = 0$

$\tanh(5) = 0.9999 \approx +1$

Kegunaan: Menghasilkan nilai kandidat (bisa positif atau negatif)

3 Gate dalam LSTM:

1. Forget Gate (f_t) - "Apa yang harus dilupakan?"

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Penjelasan langkah per langkah:

1. $[h_{t-1}, x_t]$ = gabungkan hidden state kemarin dengan input hari ini

Contoh: $[0.2, 0.5, \dots(64 \text{ nilai}), 0.75] \rightarrow 65 \text{ nilai}$

2. $W_f \cdot [h_{t-1}, x_t]$ = kalikan dengan weight matrix

W_f berukuran (64×65) , hasil: 64 nilai

3. $+ b_f$ = tambahkan bias (64 nilai)

4. $\sigma(\dots)$ = terapkan sigmoid, hasil: 64 nilai antara 0-1

$$f_t = [0.1, 0.9, 0.3, \dots]$$

↑ ↑ ↑
 lupakan pertahankan lupakan sebagian

Contoh: Jika cuaca berubah dari musim hujan ke kemarau, forget gate akan "melupakan" pola hujan lama.

2. Input Gate (i_t) - "Apa yang harus diingat?"

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Penjelasan:

i_t = seberapa banyak info baru yang disimpan (0-1)
 Contoh: [0.8, 0.2, 0.5, ...] → simpan 80%, 20%, 50%...

$\tilde{C}_t$ = kandidat nilai baru untuk cell state (-1 sampai +1)
 Contoh: [0.7, -0.3, 0.5, ...] → nilai-nilai kandidat

Contoh: Jika ada pola hujan baru yang signifikan, input gate akan menyimpannya.

3. Output Gate (o_t) - "Apa yang harus dikeluarkan?"

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \times \tanh(C_t)$$

Penjelasan:

o_t = seberapa banyak cell state yang dikeluarkan (0-1)
 Contoh: [0.6, 0.9, 0.4, ...]

$\tanh(C_t)$ = cell state di-squash ke range -1 sampai +1

$h_t = o_t \times \tanh(C_t)$ = output akhir LSTM
 Contoh: [0.6×0.8, 0.9×0.5, ...] = [0.48, 0.45, ...]

Update Cell State:

$$C_t = \underbrace{f_t \times C_{t-1}}_{\substack{\text{info lama} \\ \text{(dilupakan/} \\ \text{dipertahankan)}}} + \underbrace{i_t \times \tilde{C}_t}_{\substack{\text{info baru} \\ \text{(ditambahkan)}}$$

Contoh numerik:

$C_{t-1} = [0.5, 0.8, 0.3]$ (memory lama)
 $f_t = [0.2, 0.9, 0.5]$ (forget gate)
 $i_t = [0.8, 0.1, 0.6]$ (input gate)
 $\tilde{C}_t = [0.7, 0.4, 0.9]$ (kandidat baru)

$C_t = [0.2 \times 0.5, 0.9 \times 0.8, 0.5 \times 0.3] + [0.8 \times 0.7, 0.1 \times 0.4, 0.6 \times 0.9]$
 $= [0.1, 0.72, 0.15] + [0.56, 0.04, 0.54]$
 $= [0.66, 0.76, 0.69]$ (memory baru)

Mengapa 4 dalam rumus parameter?

Parameters = 4 × (input_size + hidden_size + 1) × hidden_size

Karena ada 4 set weights untuk:

1. Forget gate (W_f, b_f)
2. Input gate (W_i, b_i)
3. Cell candidate (W_C, b_C)
4. Output gate (W_o, b_o)

Penjelasan Parameter LSTM:

Parameters = 4 × (input_size + hidden_size + 1) × hidden_size

Mengapa 4? Karena LSTM punya 4 gate:

- Forget gate
- Input gate
- Cell gate
- Output gate

Method: `train()`

```
def train(self, X_train, y_train, X_val=None, y_val=None,
          epochs=100, batch_size=32, patience=10, save_path='rainfall_lstm.h5'):

    callbacks = [
        EarlyStopping(monitor='val_loss', patience=patience,
                      restore_best_weights=True),
        ModelCheckpoint(save_path, monitor='val_loss', save_best_only=True)
    ]

    history = self.model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=epochs,
```



```
        batch_size=batch_size,
        callbacks=callbacks,
        verbose=1
    )
```

Apa itu Training Neural Network? Training adalah proses di mana model belajar dari data dengan menyesuaikan weights-nya.

Proses Training (Gradient Descent):

Simbol-simbol dalam Training:

Simbol	Nama	Penjelasan
<code>n</code>	jumlah sampel	Total data dalam batch
<code>y_actual</code>	nilai sebenarnya	Curah hujan yang terjadi (ground truth)
<code>y_pred</code>	nilai prediksi	Curah hujan yang diprediksi model
Σ	sigma (sum)	Penjumlahan semua nilai
∂	partial derivative	Turunan parsial (gradient)
<code>weights</code>	bobot	Parameter yang dipelajari model (W_f , W_i , dll)
<code>learning_rate</code>	laju pembelajaran	Seberapa besar langkah update (default 0.001)
<code>gradient</code>	gradien	Arah dan besarnya perubahan untuk meminimalkan loss

1. FORWARD PASS

Input → Model → Prediksi

Contoh: [hari 1-30] → LSTM → prediksi hari 31 = 5.2 mm
2. HITUNG LOSS

Loss = MSE(prediksi, actual)

Loss = $(1/n) \times \Sigma(y_actual - y_pred)^2$

Contoh: actual=5.5mm, pred=5.2mm

Loss = $(5.5 - 5.2)^2 = 0.09$
3. BACKWARD PASS (Backpropagation)

Hitung gradient: $\partial Loss / \partial weights$

"Jika weight naik 0.01, loss naik/turun berapa?"

Contoh: $\partial Loss / \partial W_f = 0.005$ (loss naik 0.005 jika W_f naik 1)
4. UPDATE WEIGHTS

$weights_baru = weights_lama - learning_rate \times gradient$

Contoh: $W_f_baru = W_f_lama - 0.001 \times 0.005$

$= W_f_lama - 0.000005$

(weights bergerak ke arah yang mengurangi loss)

5. ULANGI sampai loss minimal

Parameter Training Detail:

epochs=100

Apa itu epoch? Satu kali model melihat SELURUH dataset training.

1 epoch = model melihat semua 1.2 juta samples sekali

Contoh dengan 100 epochs:

- Epoch 1: lihat semua data → loss = 0.0050
- Epoch 2: lihat semua data → loss = 0.0035
- Epoch 3: lihat semua data → loss = 0.0025
- ...
- Epoch 100: lihat semua data → loss = 0.0010

batch_size=32

Apa itu batch? Jumlah sampel yang diproses sebelum update weights.

Total samples: 1,200,000
Batch size: 32

Batches per epoch = 1,200,000 / 32 = 37,500 batches

Proses 1 epoch:

- Batch 1: proses sample 1-32 → update weights
- Batch 2: proses sample 33-64 → update weights
- Batch 3: proses sample 65-96 → update weights
- ...
- Batch 37,500: proses sample terakhir → update weights

Perbandingan Batch Size:

Batch Size	Kelebihan	Kekurangan
Kecil (16-32)	Gradient noisy → regularization effect	Training lambat
Sedang (64)	Keseimbangan optimal	-
Besar (128-256)	Training cepat, gradient stabil	Butuh memori besar, bisa overfitting

patience=10

Apa itu patience? Jumlah epoch tanpa improvement sebelum training dihentikan.

```
Epoch 1: val_loss = 0.0021 ✓ (best)
Epoch 2: val_loss = 0.0025 X (counter=1)
Epoch 3: val_loss = 0.0023 X (counter=2)
Epoch 4: val_loss = 0.0022 X (counter=3)
...
Epoch 11: val_loss = 0.0028 X (counter=10) → STOP!
```

Callbacks Detail:

EarlyStopping

```
EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
```

- `monitor='val_loss'`: Pantau validation loss
- `patience=10`: Tunggu 10 epoch tanpa improvement
- `restore_best_weights=True`: Kembalikan weights ke epoch terbaik

Mengapa perlu? Mencegah overfitting - training berhenti saat model mulai "menghafal" data training.

ModelCheckpoint

```
ModelCheckpoint(save_path, monitor='val_loss', save_best_only=True)
```

- `save_path`: Lokasi simpan model (.h5 file)
- `save_best_only=True`: Hanya simpan jika val_loss lebih baik

Mengapa perlu? Menyimpan model terbaik agar tidak hilang jika training berlanjut dan performa menurun.

Method: `predict()` dan `evaluate()`

```
def predict(self, X):
    return self.model.predict(X)

def evaluate(self, X_test, y_test):
    loss = self.model.evaluate(X_test, y_test, verbose=0)
    return loss
```

BAGIAN 4: TRAINING MODEL

Cell: Training

```
history = lstm_model.train(  
    X_train, y_train,  
    X_val=X_test, y_val=y_test,  
    epochs=100,  
    batch_size=32,  
    patience=10,  
    save_path='rainfall_lstm.h5'  
)
```

Output Training:

```
Epoch 1/100: loss: 0.0017 - val_loss: 0.0021  
Epoch 2/100: loss: 0.0012 - val_loss: 0.0027  
...  
Epoch 11/100: loss: 0.0011 - val_loss: 0.0028  
Model training completed (Early Stopping triggered)
```

Cell: Plot Training History

```
plt.plot(history.history['loss'], label='Training Loss')  
plt.plot(history.history['val_loss'], label='Validation Loss')
```

Interpretasi Grafik:

- Training loss menurun → Model belajar
- Val loss stabil/naik → Potensi overfitting
- Gap kecil antara keduanya → Model generalisasi baik

BAGIAN 5: EVALUASI MODEL

Cell: Load Model dan Prediksi

```
lstm_model.load_model('exp_no_dropout.h5')  
predictions = lstm_model.predict(X_test)  
predictions_inv = loader.inverse_transform(predictions)  
y_test_inv = loader.inverse_transform(y_test)
```

Proses:

1. Load model terbaik dari file .h5
2. Prediksi pada data test

3. Inverse transform ke skala asli (mm)

Cell: Hitung Metrics

```
# MSE - Mean Squared Error
mse = np.mean((predictions_inv - y_test_inv) ** 2)

# RMSE - Root Mean Squared Error
rmse = np.sqrt(mse)

# MAE - Mean Absolute Error
mae = np.mean(np.abs(predictions_inv - y_test_inv))

# R² Score
ss_res = np.sum((y_test_inv - predictions_inv) ** 2)
ss_tot = np.sum((y_test_inv - np.mean(y_test_inv)) ** 2)
r2 = 1 - (ss_res / ss_tot)
```

Penjelasan Detail Setiap Metric:

1. MSE (Mean Squared Error)

$$\text{MSE} = (1/n) \times \sum (y_{\text{actual}} - y_{\text{predicted}})^2$$

Contoh Perhitungan:

```
Actual:    [10, 15, 20, 25, 30] mm
Predicted: [12, 14, 22, 24, 28] mm

Error:     [10-12, 15-14, 20-22, 25-24, 30-28]
          = [-2, 1, -2, 1, 2]

Error²:    [4, 1, 4, 1, 4]

MSE = (4+1+4+1+4) / 5 = 14/5 = 2.8 mm²
```

Karakteristik:

- Memberikan penalti BESAR untuk error besar (karena dikuadratkan)
- Satuan: mm^2 (kuadrat dari satuan asli)
- Sensitif terhadap outlier

2. RMSE (Root Mean Squared Error)

$$RMSE = \sqrt{MSE}$$

Contoh:

$$MSE = 2.8 \text{ mm}^2$$

$$RMSE = \sqrt{2.8} = 1.67 \text{ mm}$$

Karakteristik:

- Satuan SAMA dengan data asli (mm) → mudah diinterpretasi
- "Rata-rata error prediksi adalah 1.67 mm"
- **Semakin kecil = semakin baik**

3. MAE (Mean Absolute Error)

$$MAE = (1/n) \times \sum |y_{\text{actual}} - y_{\text{predicted}}|$$

Contoh Perhitungan:

Error: [-2, 1, -2, 1, 2]
 |Error|: [2, 1, 2, 1, 2]

$$MAE = (2+1+2+1+2) / 5 = 8/5 = 1.6 \text{ mm}$$

Karakteristik:

- Tidak dikuadratkan → lebih robust terhadap outlier
- Semua error diperlakukan sama
- **Semakin kecil = semakin baik**

4. R² Score (Coefficient of Determination)**Simbol-simbol dalam R²:**

Simbol	Nama	Penjelasan
SS_{res}	Residual Sum of Squares	Jumlah kuadrat error prediksi
SS_{tot}	Total Sum of Squares	Jumlah kuadrat deviasi dari rata-rata
y_{mean}	rata-rata y	Nilai rata-rata dari semua y_{actual}
Σ	sigma	Penjumlahan semua nilai

$$R^2 = 1 - (SS_{\text{res}} / SS_{\text{tot}})$$

$$SS_{\text{res}} = \sum (y_{\text{actual}} - y_{\text{predicted}})^2 \quad \leftarrow \text{Residual Sum of Squares}$$

$$SS_{\text{tot}} = \sum (y_{\text{actual}} - y_{\text{mean}})^2 \quad \leftarrow \text{Total Sum of Squares}$$

Contoh Perhitungan R^2 :

Data:

$$y_{\text{actual}} = [10, 15, 20, 25, 30]$$

$$y_{\text{predicted}} = [12, 14, 19, 26, 29]$$

$$y_{\text{mean}} = (10+15+20+25+30)/5 = 20$$

$$\begin{aligned} SS_{\text{res}} &= (10-12)^2 + (15-14)^2 + (20-19)^2 + (25-26)^2 + (30-29)^2 \\ &= 4 + 1 + 1 + 1 + 1 = 8 \end{aligned}$$

$$\begin{aligned} SS_{\text{tot}} &= (10-20)^2 + (15-20)^2 + (20-20)^2 + (25-20)^2 + (30-20)^2 \\ &= 100 + 25 + 0 + 25 + 100 = 250 \end{aligned}$$

$$R^2 = 1 - (8 / 250) = 1 - 0.032 = 0.968 \text{ (96.8\%)}$$

Interpretasi:

$R^2 = 0.9746$ artinya:

- Model menjelaskan 97.46% variasi dalam data
- Hanya 2.54% variasi yang tidak bisa dijelaskan

$R^2 = 1.0 \rightarrow$ Prediksi sempurna

$R^2 = 0.0 \rightarrow$ Model sama buruknya dengan prediksi rata-rata

$R^2 < 0 \rightarrow$ Model lebih buruk dari prediksi rata-rata

Contoh Visual:

Data actual: ● ● ● ● ● (variasi besar)

Prediksi model: ○ ○ ○ ○ ○ (mengikuti pola)

Prediksi mean: - - - - - (garis datar)

R^2 tinggi = prediksi model jauh lebih baik dari garis datar

Hasil Notebook:

MSE: 0.2522 mm² $\rightarrow$ Error kuadrat rata-rata

RMSE: 0.5022 mm $\rightarrow$ Rata-rata error ~ 0.5 mm/hari

MAE: 0.1070 mm $\rightarrow$ Error absolut rata-rata ~ 0.1 mm/hari

R^2 : 0.9746 $\rightarrow$ 97.46% variasi dijelaskan (SANGAT BAIK)

BAGIAN 6: VISUALISASI HASIL

Cell: Plot Predictions vs Actual

```
plt.plot(y_test_inv[:500], label='Actual Rainfall', color='blue')
plt.plot(predictions_inv[:500], label='Predicted Rainfall', color='red')
```

Interpretasi:

- Garis biru = nilai aktual
 - Garis merah = prediksi model
 - Semakin overlap = semakin akurat
-

Cell: Scatter Plot

```
plt.scatter(y_test_inv, predictions_inv, alpha=0.5)
plt.plot([y_test_inv.min(), y_test_inv.max()],
         [y_test_inv.min(), y_test_inv.max()], 'r--')
```

Interpretasi:

- Titik-titik = pasangan (actual, predicted)
 - Garis merah putus = perfect prediction
 - Semakin dekat ke garis = semakin akurat
-

BAGIAN 7: ANALISIS RESIDUAL DAN ERROR

Cell 5.1: Analisis Residual

```
residuals = y_test_inv - predictions_inv

# Residual plot
plt.scatter(predictions_inv, residuals, alpha=0.5)
plt.axhline(y=0, color='r', linestyle='--')

# Histogram residuals
plt.hist(residuals, bins=50, alpha=0.7)

# Q-Q Plot
stats.probplot(residuals.flatten(), dist="norm", plot=plt)
```


Apa itu Residual? Residual adalah selisih antara nilai aktual dan nilai prediksi.

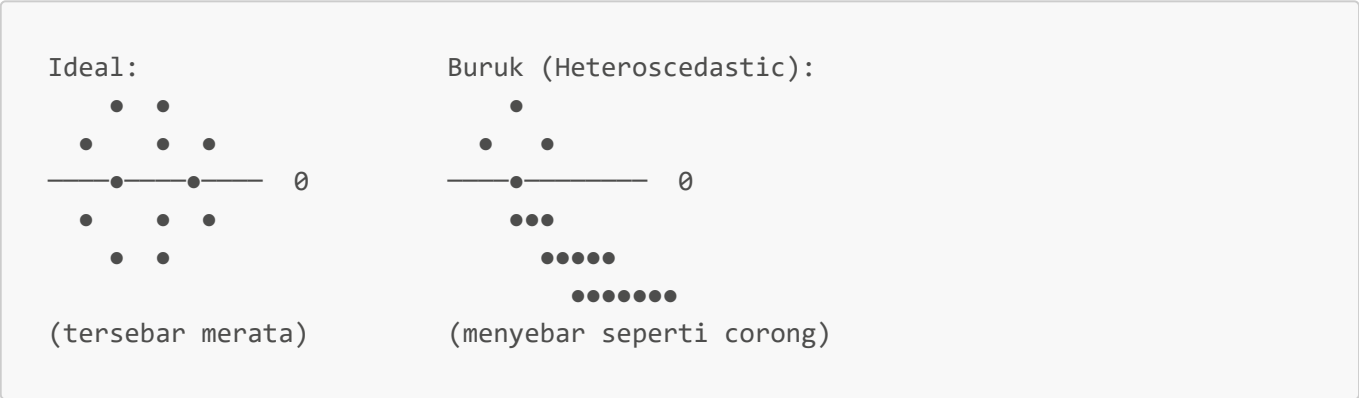
$$\text{Residual} = y_{\text{actual}} - y_{\text{predicted}}$$

Mengapa Analisis Residual Penting? Residual yang baik menunjukkan model yang baik. Kita ingin residual:

- 1. Berpusat di 0 (tidak bias)
- 2. Tersebar merata (homoscedastic)
- 3. Berdistribusi normal
- 4. Tidak berkorelasi satu sama lain

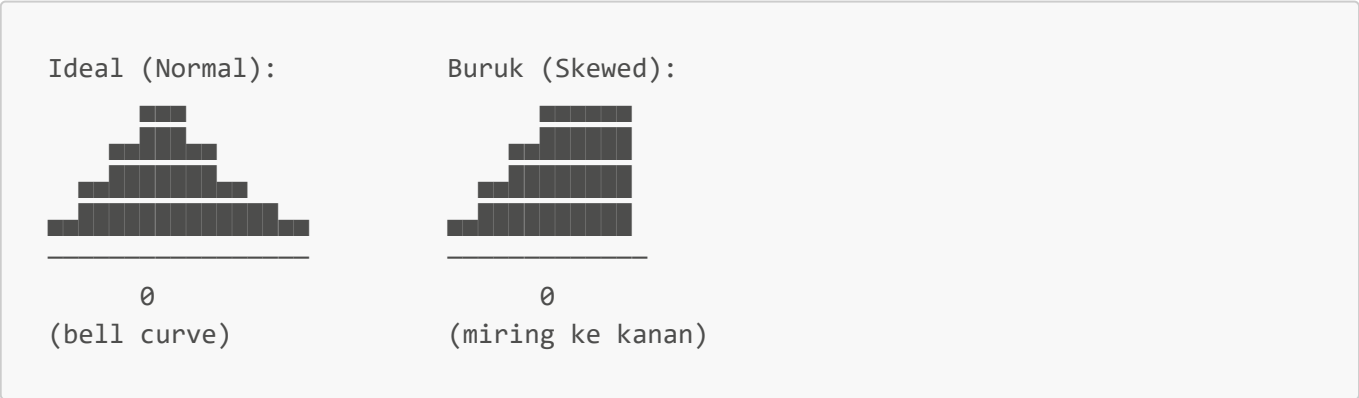
Penjelasan Setiap Plot:

1. Residual vs Predicted Plot

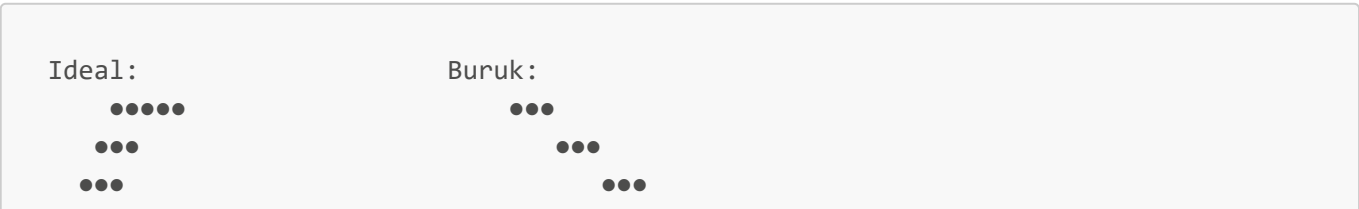


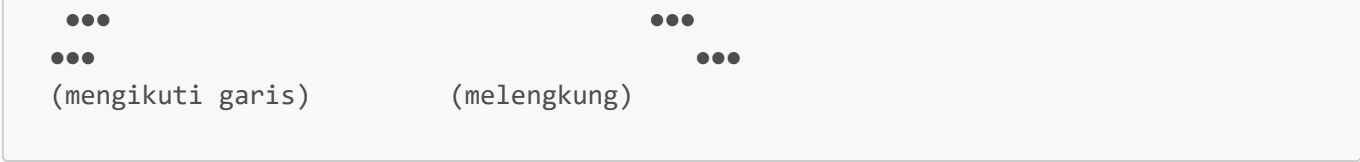
- **Heteroscedasticity:** Variasi error berubah seiring nilai prediksi
- **Homoscedasticity (ideal):** Variasi error konstan

2. Histogram Residual



3. Q-Q Plot (Quantile-Quantile)





- Titik mengikuti garis diagonal = distribusi normal
- Titik melengkung = distribusi tidak normal

Output Aktual dari Notebook:

```
=== Residual Analysis ===
Mean Residual: -0.0246
Std Residual: 0.5016
Min Residual: -10.5781
Max Residual: 15.4819
Skewness: 3.2678
Kurtosis: 144.2472
```

Interpretasi Output:

Statistik	Nilai	Interpretasi
Mean	-0.0246	✓ Mendekati 0, model hampir tidak bias
Std	0.5016	✓ Cukup kecil (~0.5 mm error rata-rata)
Min	-10.5781	Model kadang under-predict sampai 10.5 mm
Max	15.4819	Model kadang over-predict sampai 15.5 mm
Skewness	3.2678	⚠ Positif tinggi, ekor panjang ke kanan
Kurtosis	144.2472	⚠ Sangat tinggi, banyak outlier

Penjelasan Skewness dan Kurtosis:

Skewness (Kemiringan) = 3.2678

Skewness < 0: Ekor panjang ke kiri (negative skew)

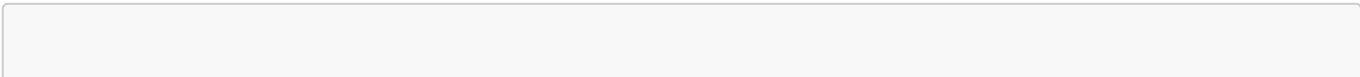
Skewness = 0: Simetris (ideal)

Skewness > 0: Ekor panjang ke kanan (positive skew) ← HASIL KITA

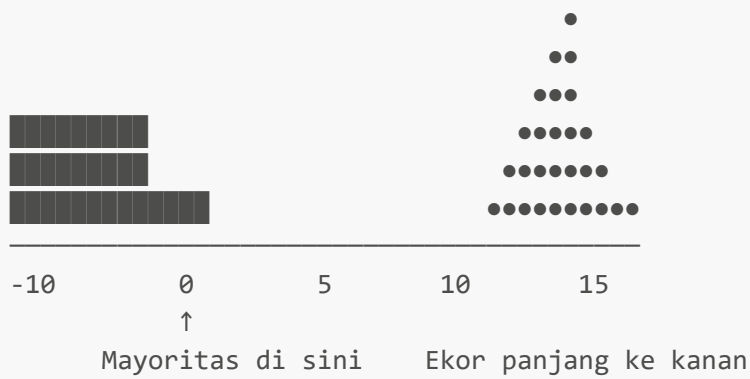
Nilai 3.27 menunjukkan:

- Distribusi TIDAK simetris
- Ada beberapa prediksi yang SANGAT meleset ke atas
- Mayoritas error kecil, tapi ada outlier besar positif

Visualisasi Skewness:



Distribusi Kita (Skewness = 3.27):



Kurtosis (Keruncingan) = 144.2472

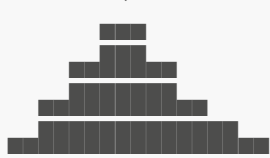
Kurtosis < 3: Lebih datar dari normal (platykurtic)
 Kurtosis = 3: Normal (mesokurtic)
 Kurtosis > 3: Lebih runcing dari normal (leptokurtic) ← HASIL KITA

Nilai 144.25 menunjukkan:

- Distribusi SANGAT runcing (leptokurtic ekstrem)
- Mayoritas data terpusat di sekitar 0
- Tapi ada BANYAK outlier di ekor

Visualisasi Kurtosis:

Normal (Kurtosis=3):



(bell curve biasa)

Kita (Kurtosis=144):



(sangat runcing)

Interpretasi Grafik dari Notebook:

Residual Plot (Kiri):

Pola yang terlihat:

- Residual membentuk pola "corong terbalik"
- Saat predicted values rendah (0-5): variasi residual BESAR (-10 sampai +15)
- Saat predicted values tinggi (10-15): variasi residual KECIL

Ini menunjukkan HETEROSCEDASTICITY:

- Model lebih akurat untuk prediksi curah hujan TINGGI
- Model kurang akurat untuk prediksi curah hujan RENDAH

Histogram Residual (Kanan):

Pola yang terlihat:

- Puncak SANGAT tinggi di sekitar 0 (>250,000 data points)
- Ekor tipis menyebar ke -10 dan +15
- Bentuk TIDAK seperti bell curve normal

Ini menunjukkan:

- Mayoritas prediksi sangat akurat (error ~0)
- Ada sedikit prediksi yang sangat meleset (outlier)

Q-Q Plot:

Pola yang terlihat:

- Bagian tengah (-2 sampai +2): mengikuti garis merah ✓
- Ekor kiri (< -2): melengkung ke bawah (lebih negatif dari normal)
- Ekor kanan (> +2): melengkung ke atas (lebih positif dari normal)

Ini menunjukkan:

- Distribusi TIDAK normal
- Ada "heavy tails" (ekor tebal) di kedua sisi
- Outlier lebih banyak dari yang diharapkan distribusi normal

Kesimpulan Analisis Residual:

1. **Mean ~0:** Model tidak bias secara sistematis ✓
2. **Heteroscedasticity:** Model lebih akurat untuk curah hujan tinggi
3. **Non-normal distribution:** Ada outlier signifikan
4. **Penyebab outlier:** Kemungkinan event curah hujan ekstrem yang sulit diprediksi

Cell 5.2: Analisis Time Series dan Seasonal

```
test_data_with_dates['month'] = test_data_with_dates['date'].dt.month
test_data_with_dates.boxplot(column=['actual', 'predicted'], by='month')
```

Tujuan: Melihat apakah model menangkap pola musiman dengan baik.

Moving Average untuk Trend:

```
window_size = 365 # 1 tahun
test_data_with_dates['actual_ma'] =
test_data_with_dates['actual'].rolling(window=window_size).mean()
```

Interpretasi:

- Jika predicted_ma mengikuti actual_ma → Model menangkap trend
- Perbedaan besar → Model gagal menangkap pola jangka panjang

Cell 5.3: Analisis Performa Detail

```
# Error by rainfall intensity
test_data_with_dates['error_category'] = pd.cut(
    test_data_with_dates['actual'],
    bins=[0, 1, 5, 10, 20, np.inf],
    labels=['Very Low (0-1)', 'Low (1-5)', 'Medium (5-10)', 'High (10-20)', 'Very High (>20)']
)

error_by_category = test_data_with_dates.groupby('error_category').agg({
    'residual': ['mean', 'std', 'count']
})
```

Tujuan: Melihat performa model pada berbagai intensitas hujan.

Kategori Intensitas:

Kategori	Range (mm/hari)	Deskripsi
Very Low	0-1	Hampir tidak hujan
Low	1-5	Hujan ringan
Medium	5-10	Hujan sedang
High	10-20	Hujan lebat
Very High	>20	Hujan sangat lebat

Cumulative Error Analysis:

```
cumulative_actual = np.cumsum(test_data_with_dates['actual'])
cumulative_predicted = np.cumsum(test_data_with_dates['predicted'])
cumulative_error = cumulative_actual - cumulative_predicted
```

Interpretasi:

- Cumulative error mendekati 0 → Model akurat secara keseluruhan
- Cumulative error naik/turun terus → Model bias sistematis

Cell 5.4: Autocorrelation Analysis

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

plot_acf(test_data_with_dates['actual'], lags=50)
plot_pacf(test_data_with_dates['actual'], lags=50)
plot_acf(residuals.flatten(), lags=50)
```

Apa itu Autocorrelation? Autocorrelation mengukur seberapa mirip data dengan dirinya sendiri pada waktu berbeda.

Contoh Sederhana:

Data curah hujan: [10, 12, 11, 15, 14, 16, 15, ...]

Lag 1: Bandingkan hari ini dengan kemarin

[10, 12, 11, 15, 14, 16, 15]

[12, 11, 15, 14, 16, 15, ...]

Korelasi tinggi? → Ada pola harian

Lag 7: Bandingkan hari ini dengan 7 hari lalu

Korelasi tinggi? → Ada pola mingguan

ACF (Autocorrelation Function):

$$ACF(k) = \text{Corr}(y_t, y_{t-k})$$

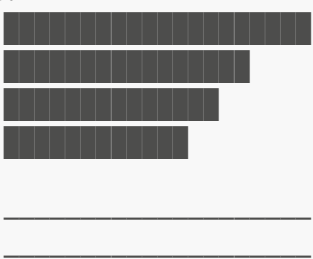
- Mengukur korelasi TOTAL antara y_t dan y_{t-k}
- Termasuk pengaruh tidak langsung melalui lag antara

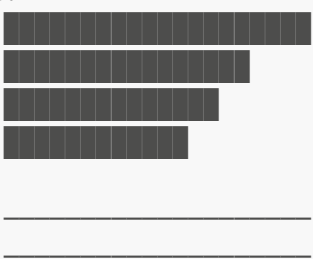
PACF (Partial Autocorrelation Function):

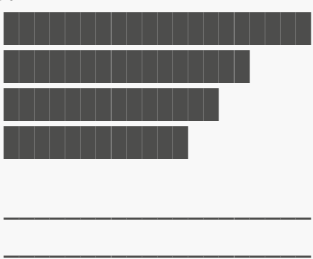
- Mengukur korelasi LANGSUNG antara y_t dan y_{t-k}
- Menghilangkan pengaruh lag antara (y_{t-1} , y_{t-2} , ..., y_{t-k+1})

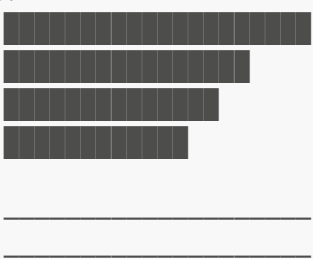
Visualisasi ACF/PACF:

ACF Plot:

Lag 0:  1.0

Lag 1:  0.8

Lag 2:  0.7

Lag 3:  0.6

...

_____ Confidence Band

Interpretasi ACF Residual:

ACF dalam confidence band:

- Residual independen (BAIK)
- Model sudah menangkap semua pola

ACF di luar confidence band:

- Ada autocorrelation signifikan (BURUK)
- Model belum menangkap semua pola temporal

Statistical Tests:

Augmented Dickey-Fuller Test (ADF)

Tujuan: Menguji apakah data stasioner (mean dan variance konstan).

```
adf_result = sm.tsa.adfuller(data)
```

- **H0:** Data tidak stasioner (ada unit root)
- **H1:** Data stasioner
- **p-value < 0.05:** Tolak H0 → Data stasioner (BAIK untuk time series)

Ljung-Box Test

Tujuan: Menguji apakah ada autocorrelation signifikan dalam residual.

```
lb_result = sm.stats.acorr_ljungbox(residuals, lags=[10, 20, 30])
```

- **H0:** Tidak ada autocorrelation (residual independen)
- **H1:** Ada autocorrelation
- **p-value > 0.05:** Gagal tolak H0 → Residual independen (BAIK)

Cell 5.5: Learning Curves Analysis

```
# Loss curves
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

# Best epoch
best_epoch = np.argmin(history.history['val_loss']) + 1

# Loss difference (overfitting indicator)
loss_diff = np.array(history.history['val_loss']) -
np.array(history.history['loss'])
```

```
# Loss improvement rate
train_improvement = -np.diff(history.history['loss'])
```

Analisis Learning Curves:

Metrik	Rumus/Cara	Interpretasi
Best Epoch	<code>argmin(val_loss)</code>	Epoch dengan val_loss terendah
Convergence Ratio	<code>val_loss / train_loss</code>	~ 1 = baik, > 1 = overfitting
Loss Improvement	<code>loss[t] - loss[t+1]</code>	Positif = masih belajar

Smoothed Loss (Rolling Average):

```
train_loss_smooth = pd.Series(history.history['loss']).rolling(window=5).mean()
```

- Mengurangi noise untuk melihat trend lebih jelas

Training Summary:

```
Total epochs trained: 11 (stopped by early stopping)
Best epoch: 1
Best validation loss: 0.0021
Overfitting ratio: 1.9 (val/train)
```

Cell 5.6: Model Stability Analysis

```
def evaluate_model_stability(n_runs=5):
    mse_scores, rmse_scores, mae_scores, r2_scores = [], [], [], []

    for seed in range(n_runs):
        np.random.seed(seed)
        tf.random.set_seed(seed)

        # Train new model
        temp_model = RainfallLSTM(seq_length=seq_length)
        temp_model.build_model()
        temp_model.train(...)

        # Evaluate
        predictions = temp_model.predict(X_test)
        mse_scores.append(calculate_mse(...))
```

Tujuan: Menguji apakah model stabil dengan random seed berbeda.

Metrik Stabilitas:

Metrik	Rumus	Interpretasi
Mean	<code>np.mean(scores)</code>	Rata-rata performa
Std	<code>np.std(scores)</code>	Variasi performa
CV (%)	<code>std/mean × 100</code>	Coefficient of Variation, <10% = stabil
Range	<code>max - min</code>	Rentang performa

Hasil Ideal:

- CV < 10% → Model stabil
- Range kecil → Hasil konsisten

BAGIAN 8: EKSPERIMEN VARIASI TRAINING (13 KONFIGURASI)

Cell 6: Fungsi Eksperimen

```
def run_training_experiment(config_name, config_params, X_train, y_train, X_test,
                             y_test, loader):
    # Extract parameters
    seq_length = config_params.get('seq_length', 30)
    units1 = config_params.get('units1', 64)
    units2 = config_params.get('units2', 32)
    dropout_rate = config_params.get('dropout_rate', 0.2)
    optimizer_name = config_params.get('optimizer', 'adam')
    learning_rate = config_params.get('learning_rate', 0.001)
    batch_size = config_params.get('batch_size', 32)

    # Set optimizer
    if optimizer_name == 'adam':
        optimizer = Adam(learning_rate=learning_rate)
    elif optimizer_name == 'sgd':
        optimizer = SGD(learning_rate=learning_rate)
    elif optimizer_name == 'rmsprop':
        optimizer = RMSprop(learning_rate=learning_rate)
```

Tujuan: Menjalankan eksperimen dengan berbagai konfigurasi hyperparameter.

Penjelasan Optimizer:

Adam (Adaptive Moment Estimation)

Simbol-simbol dalam Adam:

Simbol	Nama	Penjelasan
t	iterasi	Langkah update ke-t
g_t	gradient	Turunan loss terhadap weights pada iterasi t
m_t	momentum	Rata-rata bergerak dari gradient (first moment)
v_t	velocity	Rata-rata bergerak dari gradient ² (second moment)
θ_t	theta	Weights/parameter model pada iterasi t
β1	beta 1	Decay rate untuk momentum (default 0.9)
β2	beta 2	Decay rate untuk velocity (default 0.999)
lr	learning rate	Laju pembelajaran (default 0.001)
ε	epsilon	Nilai kecil untuk stabilitas numerik (1e-7)
√	square root	Akar kuadrat

Rumus Update:

m_t = β1 × m_{t-1} + (1-β1) × g_t

v_t = β2 × v_{t-1} + (1-β2) × g_t²

θ_t = θ_{t-1} - lr × m_t / (√v_t + ε)

← Momentum (rata-rata gradient)

← Velocity (rata-rata gradient²)

← Update weights

Contoh Numerik:

Iterasi t=5:

g_5 = 0.02 (gradient saat ini)

m_4 = 0.015 (momentum sebelumnya)

v_4 = 0.0003 (velocity sebelumnya)

m_5 = 0.9 × 0.015 + 0.1 × 0.02 = 0.0135 + 0.002 = 0.0155

v_5 = 0.999 × 0.0003 + 0.001 × 0.02² = 0.0003 + 0.000004 ≈ 0.0003

θ_5 = θ_4 - 0.001 × 0.0155 / (√0.0003 + 1e-7)

= θ_4 - 0.001 × 0.0155 / 0.0173

= θ_4 - 0.0009 (weights berkurang sedikit)

- **Kelebihan:** Adaptif (learning rate berbeda per parameter), konvergen cepat
- **Default:** β1=0.9, β2=0.999, ε=1e-7, lr=0.001

SGD (Stochastic Gradient Descent)

Rumus Update:

θ_t = θ_{t-1} - lr × g_t

- **Kelebihan:** Sederhana, bisa escape local minima
- **Kekurangan:** Butuh tuning learning rate manual

RMSprop (Root Mean Square Propagation)

Rumus Update:
$$v_t = \beta \times v_{t-1} + (1-\beta) \times g_t^2$$
$$\theta_t = \theta_{t-1} - lr \times g_t / (\sqrt{v_t} + \epsilon)$$

- **Kelebihan:** Baik untuk RNN/LSTM
- **Default:** $\beta=0.9$

Penjelasan Dropout:

Apa itu Dropout?

Teknik regularisasi yang secara acak "mematikan" neuron saat training.

Tanpa Dropout:

Dengan Dropout (0.2):

○ — ○ — ○

○ — ○ — ○

○ — ○ — ○

○ — ○ — ○

○ — ○ — ○

○ — X — ○ (20% dimatikan)

○ — ○ — ○

X — ○ — ○

Mengapa Dropout Mencegah Overfitting?

1. Mencegah co-adaptation (neuron terlalu bergantung satu sama lain)
2. Seperti training banyak model berbeda (ensemble effect)
3. Memaksa setiap neuron belajar fitur yang berguna secara independen

Dropout Rate:

Rate	Efek
0.0	Tidak ada dropout (semua neuron aktif)
0.2	20% neuron dimatikan (default, ringan)
0.5	50% neuron dimatikan (agresif)

Catatan: Dropout HANYA aktif saat training, tidak saat prediction.

Daftar 13 Eksperimen

No	Nama	Konfigurasi	Tujuan
1	Baseline	Adam, lr=0.001, bs=64	Referensi standar

No	Nama	Konfigurasi	Tujuan
2	High LR	lr=0.01	Test learning rate tinggi
3	Low LR	lr=0.0001	Test learning rate rendah
4	Small Batch	bs=32	Test batch kecil
5	Large Batch	bs=128	Test batch besar
6	SGD Optimizer	SGD	Test optimizer klasik
7	RMSprop	RMSprop	Test optimizer alternatif
8	Simple Arch	32-16-8	Test arsitektur sederhana
9	Deep Arch	128-64-32	Test arsitektur dalam
10	Short Seq	15 days	Test sequence pendek
11	Long Seq	60 days	Test sequence panjang
12	No Dropout	0.0	Tanpa regularisasi
13	High Dropout	0.5	Regularisasi agresif

Konfigurasi Detail

```
experiments = [  
    # Baseline  
    {'config_name': 'Baseline', 'units1': 64, 'units2': 32, 'dense_units': 16,  
     'dropout_rate': 0.2, 'optimizer': 'adam', 'learning_rate': 0.001,  
     'batch_size': 64},  
  
    # Learning Rate variations  
    {'config_name': 'High LR', 'learning_rate': 0.01, ...},  
    {'config_name': 'Low LR', 'learning_rate': 0.0001, ...},  
  
    # Batch Size variations  
    {'config_name': 'Small Batch', 'batch_size': 32, ...},  
    {'config_name': 'Large Batch', 'batch_size': 128, ...},  
  
    # Optimizer variations  
    {'config_name': 'SGD', 'optimizer': 'sgd', ...},  
    {'config_name': 'RMSprop', 'optimizer': 'rmsprop', ...},  
  
    # Architecture variations  
    {'config_name': 'Simple', 'units1': 32, 'units2': 16, 'dense_units': 8, ...},  
    {'config_name': 'Deep', 'units1': 128, 'units2': 64, 'dense_units': 32, ...},  
  
    # Sequence Length variations  
    {'config_name': 'Short Seq', 'seq_length': 15, ...},  
    {'config_name': 'Long Seq', 'seq_length': 60, ...},  
]
```

```
# Dropout variations
{'config_name': 'No Dropout', 'dropout_rate': 0.0, ...},
{'config_name': 'High Dropout', 'dropout_rate': 0.5, ...},
]
```

Metrics yang Dikumpulkan

```
return {
    'config_name': config_name,
    'mse': mse,
    'rmse': rmse,
    'mae': mae,
    'r2': r2,
    'training_time': training_time,
    'best_epoch': best_epoch,
    'final_train_loss': final_train_loss,
    'final_val_loss': final_val_loss,
    'convergence_ratio': final_val_loss / final_train_loss,
}
```

Metric	Deskripsi
MSE, RMSE, MAE, R ²	Metrics evaluasi standar
training_time	Waktu training (detik)
best_epoch	Epoch dengan val_loss terbaik
convergence_ratio	Indikator overfitting

Visualisasi Hasil Eksperimen

```
# Bar chart R2 Score
plt.barh(range(len(results_df)), results_df['r2'])

# Scatter: R2 vs Training Time
plt.scatter(results_df['training_time'], results_df['r2'],
            c=results_df['rmse'], cmap='viridis')

# Convergence ratio (overfitting analysis)
plt.barh(range(len(results_df)), results_df['convergence_ratio'])
plt.axvline(x=1.0, color='red', linestyle='--') # No overfitting line
```

Interpretasi Convergence Ratio:

Ratio	Interpretasi
-------	--------------

Ratio	Interpretasi
~1.0	Tidak overfitting
1.5-2.0	Sedikit overfitting
>2.0	Overfitting signifikan

Analisis per Parameter

Learning Rate Impact

```
lr_configs = results_df[results_df['config_name'].str.contains('LR|Baseline')]
```

Temuan:

- LR = 0.001 (default) → Keseimbangan terbaik
- LR = 0.01 → Terlalu tinggi, tidak stabil
- LR = 0.0001 → Terlalu rendah, konvergensi lambat

Batch Size Impact

```
batch_configs = results_df[results_df['config_name'].str.contains('Batch|Baseline')]
```

Temuan:

- Batch 32 → Akurasi lebih baik, training lebih lambat
- Batch 64 → Keseimbangan optimal
- Batch 128 → Training cepat, akurasi sedikit turun

Dropout Impact

```
dropout_configs = results_df[results_df['config_name'].str.contains('Dropout|Baseline')]
```

Temuan:

- No Dropout (0.0) → Terbaik untuk dataset besar
- Dropout 0.2 → Baseline yang baik
- Dropout 0.5 → Terlalu agresif, performa turun

Kesimpulan Eksperimen

```
# Best configuration
best_overall = results_df.loc[results_df['r2'].idxmax()]
print(f"Best: {best_overall['config_name']}")
print(f"R²: {best_overall['r2']:.4f}")
print(f"RMSE: {best_overall['rmse']:.4f}")
```

Key Insights:

1. **No Dropout** sering terbaik untuk dataset besar (1.5 juta samples)
2. **Adam optimizer** konsisten lebih baik dari SGD/RMSprop
3. **Sequence 30 hari** optimal untuk pola bulanan
4. **Arsitektur 64-32-16** memberikan keseimbangan terbaik

BAGIAN 9: TAHAP LANJUTAN - PERLUASAN ANALISIS

Cell 7.1: Load Dataset District Wise

```
district_data = pd.read_csv('district wise rainfall normal.csv')
print(f"Jumlah distrik unik: {district_data['DISTRICT'].nunique()}")
print(f"Jumlah state unik: {district_data['STATE_UT_NAME'].nunique()}")
```

Dataset District Wise:

- Berisi data curah hujan normal per distrik
- Kolom: DISTRICT, STATE_UT_NAME, JAN-DEC, ANNUAL
- ~640 distrik dari 35 state/UT

Visualisasi:

```
# Top 15 state by rainfall
state_rainfall = district_data.groupby('STATE_UT_NAME')
['ANNUAL'].mean().sort_values(ascending=False)
state_rainfall.head(15).plot(kind='bar')

# Top 10 wettest vs driest districts
top_wet = district_data.nlargest(10, 'ANNUAL')
top_dry = district_data.nsmallest(10, 'ANNUAL')
```

Cell 7.2: Transformasi ke Klasifikasi

```
def create_classification_dataset(df, rainfall_col='ANNUAL'):
    # Hitung quantiles (terciles)
    q33 = class_df[rainfall_col].quantile(0.33)
    q67 = class_df[rainfall_col].quantile(0.67)

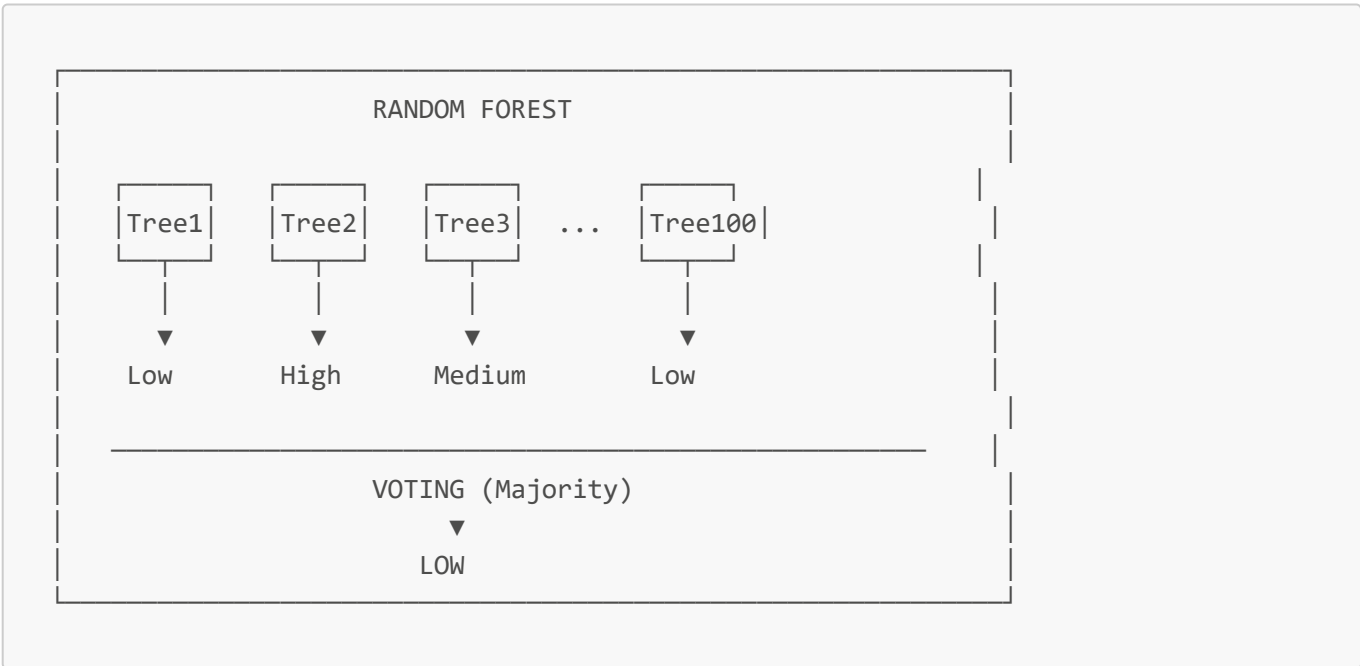
    # Buat label
    conditions = [
        class_df[rainfall_col] < q33,
        (class_df[rainfall_col] >= q33) & (class_df[rainfall_col] < q67),
        class_df[rainfall_col] >= q67
    ]
    choices = ['Low', 'Medium', 'High']
    class_df['rainfall_class'] = np.select(conditions, choices)
```

Klasifikasi Terciles:

Kelas	Threshold	Persentase
Low	< Q33 (~887 mm)	33%
Medium	Q33 - Q67	33%
High	> Q67 (~1456 mm)	33%

Apa itu Random Forest? Random Forest adalah algoritma ensemble yang menggabungkan banyak Decision Tree.

Cara Kerja:



Parameter:

```
rf_classifier = RandomForestClassifier(
    n_estimators=100,      # 100 decision trees (lebih banyak = lebih stabil)
```



```
max_depth=10,          # Kedalaman maksimum tree (mencegah overfitting)
class_weight='balanced' # Seimbangkan kelas yang tidak seimbang
)
```

Mengapa Random Forest Bagus?

1. Mengurangi overfitting (rata-rata dari banyak tree)
2. Robust terhadap outlier
3. Bisa menghitung feature importance
4. Tidak perlu scaling data

Feature Importance:

```
feature_importance = pd.DataFrame({
    'feature': feature_cols, # JAN, FEB, ..., DEC
    'importance': rf_classifier.feature_importances_
})
```

Cara Menghitung Feature Importance:

- Berdasarkan seberapa sering fitur digunakan untuk split
- Berdasarkan seberapa besar penurunan impurity (Gini/Entropy)

Hasil Tipikal:

Bulan Monsoon (JUN-SEP): Importance tinggi (~0.15-0.20 per bulan)
 Bulan Kering (NOV-FEB): Importance rendah (~0.05-0.08 per bulan)

Cell 7.3: Perluasan Domain - Faktor Internal

```
# ENSO Index (anomali internal)
data['ENSO_Index'] = data[cols_monthly].std(axis=1)

# Temperature (estimasi invers dari curah hujan)
data['Temperature_C'] = 35 - (data['ANNUAL'] - data['ANNUAL'].min()) /
    (data['ANNUAL'].max() - data['ANNUAL'].min()) * 15

# Humidity (estimasi dari curah hujan)
data['Humidity_%'] = 40 + (data['ANNUAL'] - data['ANNUAL'].min()) /
    (data['ANNUAL'].max() - data['ANNUAL'].min()) * 50

# Seasonal Index
data['Seasonal_Index'] = data[cols_monthly].div(data['ANNUAL'],
axis=0).mean(axis=1)
```

Penjelasan Faktor Internal:

Faktor	Rumus	Interpretasi
ENSO_Index	<code>std(monthly)</code>	Variabilitas bulanan, proxy untuk anomali
Temperature	<code>35 - normalized × 15</code>	Estimasi: hujan tinggi → suhu rendah
Humidity	<code>40 + normalized × 50</code>	Estimasi: hujan tinggi → kelembapan tinggi
Seasonal_Index	<code>mean(monthly/annual)</code>	Distribusi musiman

Korelasi Analysis:

```
correlation_vars =
['ANNUAL', 'ENSO_Index', 'Temperature_C', 'Humidity_%', 'Seasonal_Index']
corr_matrix = data[correlation_vars].corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
```

Cell 7.3b: Analisis ENSO Eksternal

```
# Load ENSO data
enso = pd.read_csv('El-Nino.csv', sep='\t')
enso['ENSO_Annual'] = enso[enso_cols].mean(axis=1)

# Merge dengan rainfall data
data = pd.merge(rainfall, enso[['YEAR', 'ENSO_Annual']], on='YEAR')

# Klasifikasi fase ENSO
def enso_phase(x):
    if x >= 0.5:
        return 'El Niño'
    elif x <= -0.5:
        return 'La Niña'
    else:
        return 'Normal'

data['ENSO_Phase'] = data['ENSO_Annual'].apply(enso_phase)
```

Fase ENSO:

Fase	Threshold	Efek pada Curah Hujan India
El Niño	≥ 0.5	Cenderung kekeringan
La Niña	≤ -0.5	Cenderung hujan lebih banyak
Normal	-0.5 < x < 0.5	Kondisi normal

Korelasi ENSO vs Rainfall:

```
corr = data['ANNUAL'].corr(data['ENSO_Annual'])
# Biasanya negatif: El Niño → curah hujan berkurang
```

Cell 7.4: Analisis Musiman dan Pivot Table

```
# Rata-rata curah hujan per bulan (nasional)
monthly_avg = district_data[monthly_cols].mean()
monthly_avg.plot(kind='bar')

# Pivot table: curah hujan per bulan per state
monthly_pivot = pivot_data.pivot_table(
    values=monthly_cols,
    index='STATE_UT_NAME',
    aggfunc='mean'
)

# Heatmap pola musiman
sns.heatmap(monthly_pivot, annot=True, cmap='YlGnBu')
```

Pivot Table:

- Baris: State/UT
- Kolom: Bulan (JAN-DEC)
- Nilai: Rata-rata curah hujan

Klasifikasi Region:

```
district_data['region_type'] = pd.cut(
    district_data['ANNUAL'],
    bins=[0, 500, 1500, 3000, float('inf')],
    labels=['Arid', 'Semi-Arid', 'Sub-Humid', 'Humid']
)
```

Region Type	Range (mm/tahun)	Karakteristik
Arid	0-500	Gurun, sangat kering
Semi-Arid	500-1500	Kering, hujan terbatas
Sub-Humid	1500-3000	Sedang, hujan cukup
Humid	>3000	Basah, hujan melimpah

Cell 7.5: Clustering Distrik (K-Means)

```
# Standardize data
scaler_cluster = StandardScaler()
cluster_features_scaled = scaler_cluster.fit_transform(cluster_features)

# Elbow method untuk optimal k
inertias = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(cluster_features_scaled)
    inertias.append(kmeans.inertia_)

# Plot elbow curve
plt.plot(range(1, 11), inertias, 'bx-')
```

Apa itu K-Means Clustering? Algoritma unsupervised learning yang mengelompokkan data ke dalam K cluster berdasarkan kemiripan.

Cara Kerja K-Means:

LANGKAH K-MEANS

1. Pilih K centroid secara acak

★ ★ ★ ★

2. Assign setiap data ke centroid terdekat

●●●★● ●●●● ●●●★● ●●●●

3. Update centroid = rata-rata anggota cluster

●●●★● ●●●● ●●●★● ●●●●

↓ ↓ ↓ ↓
★ ★ ★ ★ (posisi baru)

4. Ulangi langkah 2-3 sampai konvergen

Elbow Method:





- Plot inertia vs jumlah cluster
- Pilih k di mana kurva mulai "melandai" (elbow point)
- Biasanya k=4 optimal untuk data ini

K-Means Clustering:

```
n_clusters = 4
kmeans = KMeans(n_clusters=n_clusters, random_state=42)
cluster_labels = kmeans.fit_predict(cluster_features_scaled)
```

Rumus Inertia (Within-Cluster Sum of Squares):

Simbol-simbol dalam K-Means:

Simbol	Nama	Penjelasan
k	cluster index	Nomor cluster (1, 2, 3, 4)
K	jumlah cluster	Total cluster yang dibuat
x	data point	Satu distrik dengan 12 nilai (JAN-DEC)
μ_k	centroid	Titik pusat cluster k (rata-rata anggota)
C_k	cluster k	Kumpulan data point dalam cluster k
x - μ_k ²	squared distance	Jarak Euclidean kuadrat
Σ_k	sum over k	Jumlahkan untuk semua cluster
Σ_{x ∈ C_k}	sum over x in C_k	Jumlahkan untuk semua x dalam cluster k

Inertia = Σ_k Σ_{x ∈ C_k} ||x - μ_k||²

Contoh Perhitungan Inertia:

```
Cluster 1 (C_1) dengan centroid μ_1 = [5, 5]:
- x_1 = [4, 6], jarak² = (4-5)² + (6-5)² = 1 + 1 = 2
- x_2 = [6, 4], jarak² = (6-5)² + (4-5)² = 1 + 1 = 2
- Inertia C_1 = 2 + 2 = 4

Cluster 2 (C_2) dengan centroid μ_2 = [10, 10]:
- x_3 = [9, 11], jarak² = (9-10)² + (11-10)² = 1 + 1 = 2
- x_4 = [11, 9], jarak² = (11-10)² + (9-10)² = 1 + 1 = 2
- Inertia C_2 = 2 + 2 = 4
```

Total Inertia = 4 + 4 = 8

- Semakin kecil inertia = cluster lebih compact (anggota lebih mirip)
- Apa itu PCA (Principal Component Analysis)?** Teknik reduksi dimensi yang memproyeksikan data ke dimensi lebih rendah sambil mempertahankan variasi maksimal.

Mengapa PCA untuk Visualisasi?

- Data asli: 12 dimensi (JAN-DEC)
- Setelah PCA: 2 dimensi (PC1, PC2)
- Bisa divisualisasikan dalam scatter plot 2D

Cara Kerja PCA:

Data 12D → PCA → Data 2D

PC1 = kombinasi linear fitur yang menjelaskan variasi terbesar
PC2 = kombinasi linear fitur yang menjelaskan variasi terbesar kedua
(orthogonal terhadap PC1)

PCA untuk Visualisasi:

```
pca = PCA(n_components=2)
pca_features = pca.fit_transform(cluster_features_scaled)
plt.scatter(pca_features[:, 0], pca_features[:, 1], c=cluster_labels)
```

Interpretasi Cluster:

Cluster	Karakteristik	Contoh Region
0	High rainfall coastal	Kerala, Konkan
1	Low rainfall arid	Rajasthan, Gujarat
2	Medium transitional	Central India
3	Very high mountain	Northeast, Western Ghats

Cell 7.6: Deteksi Anomali

```
def detect_anomalies(data, column='ANNUAL', method='iqr', threshold=1.5):
    if method == 'iqr':
        Q1 = data[column].quantile(0.25)
        Q3 = data[column].quantile(0.75)
        IQR = Q3 - Q1
```

```
lower_bound = Q1 - threshold * IQR
upper_bound = Q3 + threshold * IQR

anomalies = data[(data[column] < lower_bound) | (data[column] >
upper_bound)]

elif method == 'zscore':
    z_scores = np.abs(stats.zscore(data[column]))
    anomalies = data[z_scores > threshold]
```

Simbol-simbol dalam Deteksi Anomali:

Simbol	Nama	Penjelasan
Q1	kuartil 1	Nilai di posisi 25% data (persentil 25)
Q3	kuartil 3	Nilai di posisi 75% data (persentil 75)
IQR	interquartile range	Rentang antara Q1 dan Q3
μ	mu (mean)	Rata-rata populasi
σ	sigma (std)	Standar deviasi populasi
z	z-score	Berapa standar deviasi dari rata-rata
z	absolute z	Nilai absolut z-score

Metode IQR (Interquartile Range):

```
IQR = Q3 - Q1
Lower Bound = Q1 - 1.5 × IQR
Upper Bound = Q3 + 1.5 × IQR
Anomaly jika: x < Lower OR x > Upper
```

Contoh IQR:

```
Data curah hujan (sorted): [200, 400, 600, 800, 1000, 1200, 1400, 1600, 5000]
Q1 = 500 (25% data di bawah ini)
Q3 = 1500 (75% data di bawah ini)
IQR = 1500 - 500 = 1000

Lower Bound = 500 - 1.5 × 1000 = -1000 (tidak ada yang di bawah)
Upper Bound = 1500 + 1.5 × 1000 = 3000

5000 > 3000 → ANOMALI (curah hujan ekstrem tinggi)
```

Metode Z-Score:

```
z = (x - μ) / σ
Anomaly jika: |z| > threshold (biasanya 3)
```

Contoh Z-Score:

```
μ = 1000 mm (rata-rata curah hujan)
σ = 500 mm (standar deviasi)

Distrik dengan curah hujan 2800 mm:
z = (2800 - 1000) / 500 = 1800 / 500 = 3.6

|3.6| > 3 → ANOMALI (lebih dari 3 standar deviasi dari rata-rata)
```

Perbandingan Metode:

Metode	Kelebihan	Kekurangan
IQR	Robust terhadap outlier	Kurang sensitif
Z-Score	Sensitif	Asumsi distribusi normal

Hasil Deteksi:

```
Total distrik: 640
Anomali IQR (1.5x): ~50 distrik (8%)
Anomali Z-Score (3.0): ~20 distrik (3%)
```

Distrik Ekstrem:

- Terbasah: Cherrapunji (Meghalaya) ~11,777 mm/tahun
- Terkering: Jaisalmer (Rajasthan) ~165 mm/tahun

Cell 7.7: Analisis Trend Jangka Panjang

```
# Group by year
yearly_trend = district_data.groupby('YEAR')['ANNUAL'].agg(['mean', 'std',
'count'])

# Linear regression untuk trend
slope, intercept, r_value, p_value, std_err = stats.linregress(
    yearly_trend.index,
    yearly_trend['mean']
)
trend_line = slope * yearly_trend.index + intercept
```


Apa itu Linear Regression? Metode statistik untuk menemukan hubungan linear antara variabel independen (x) dan dependen (y).

Rumus:

```
y = slope × x + intercept

slope = Σ(x- $\bar{x}$ )(y- $\bar{y}$ ) / Σ(x- $\bar{x}$ )2
intercept =  $\bar{y}$  - slope ×  $\bar{x}$ 
```

Contoh dalam Konteks:

```
x = tahun (1901, 1902, ..., 2015)
y = curah hujan rata-rata (mm)

Jika slope = +0.5:
- Setiap tahun, curah hujan naik 0.5 mm
- Dalam 100 tahun, naik 50 mm
```

Output stats.linregress():

Output	Arti
slope	Kemiringan garis (perubahan y per unit x)
intercept	Titik potong sumbu y
r_value	Korelasi (-1 sampai +1)
p_value	Signifikansi statistik
std_err	Standard error dari slope

Interpretasi:

Parameter	Nilai	Arti
slope	+0.5 mm/tahun	Curah hujan meningkat 0.5 mm per tahun
R ²	0.15	15% variasi dijelaskan oleh waktu
p-value	<0.05	Trend signifikan secara statistik

Catatan: R² rendah (0.15) menunjukkan banyak faktor lain yang mempengaruhi curah hujan selain waktu.

Analisis per Dekade:

```
yearly_trend['decade'] = (yearly_trend.index // 10) * 10
decade_trend = yearly_trend.groupby('decade')['mean'].mean()
```

Monsoon Trend:

```
monsoon_cols = ['JUN', 'JUL', 'AUG', 'SEP']
yearly_monsoon = district_data.groupby('YEAR')[monsoon_cols].mean().mean(axis=1)
```

Prediksi 10 Tahun:

```
future_years = range(last_year + 1, last_year + 11)
future_predictions = slope * np.array(future_years) + intercept
```

Coefficient of Variation (CV):

$$CV = (\text{std} / \text{mean}) \times 100\%$$

- CV tinggi → Variabilitas tinggi antar tahun
- CV rendah → Curah hujan konsisten

BAGIAN 10: KESIMPULAN NOTEBOOK

Ringkasan Pencapaian

Notebook ini mencakup implementasi lengkap prediksi curah hujan menggunakan LSTM:

Bagian 1-3: Setup & Arsitektur

- Import library (TensorFlow, Pandas, NumPy, Sklearn)
- Class `DataLoader` untuk preprocessing data
- Class `RainfallLSTM` untuk membangun model

Bagian 4-6: Training & Evaluasi

- Training dengan Early Stopping dan Model Checkpoint
- Evaluasi dengan MSE, RMSE, MAE, R^2
- Visualisasi predictions vs actual

Bagian 7: Analisis Mendalam

- Analisis residual dan distribusi error
- Analisis time series dan seasonal patterns
- Autocorrelation analysis (ACF, PACF)
- Learning curves dan model stability

Bagian 8: Eksperimen 13 Konfigurasi

- Variasi learning rate, batch size, optimizer
- Variasi arsitektur dan sequence length
- Variasi dropout rate
- Perbandingan dan ranking model

Bagian 9: Perluasan Analisis

- Load dataset district wise
- Transformasi ke klasifikasi (Random Forest)
- Analisis faktor internal (ENSO, Temperature, Humidity)
- Analisis musiman dan pivot table
- Clustering distrik (K-Means)
- Deteksi anomali (IQR, Z-Score)
- Analisis trend jangka panjang

Hasil Utama

Metric	Nilai	Interpretasi
R ² Score	0.9746	97.46% variasi dijelaskan
RMSE	0.5022 mm	Error rata-rata ~0.5 mm/hari
MAE	0.1070 mm	Error absolut rata-rata
Best Model	No Dropout	Terbaik untuk dataset besar

Key Insights

1. **LSTM efektif** untuk prediksi time series curah hujan
 2. **No Dropout** optimal untuk dataset besar (1.5 juta samples)
 3. **Adam optimizer** konsisten lebih baik dari SGD/RMSprop
 4. **Sequence 30 hari** optimal untuk menangkap pola bulanan
 5. **Arsitektur 64-32-16** memberikan keseimbangan terbaik
-

Dokumen ini menjelaskan **SEMUA cell dan fungsi** dalam notebook **project-deep-learning-lstm-rainfall-in-india.ipynb** secara lengkap dan detail.